

# Reviewer Pack — Minimal Geometric Entropy of Planar Graphs and Communication...

Entry 04312cc1eeef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:59:14 UTC

## Minimal Geometric Entropy of Planar Graphs and Communication Complexity Rank

**Entry ID:** 04312cc1eeef **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:59:14 UTC

### 1. Conjecture

**Field A** (mathematical branch): Geometric Information Theory (Entropy) **Field B** (complexity object): Communication Complexity

#### Statement:

For every planar graph  $G$ , the minimal geometric entropy  $H(G)$  of its vertex set is linearly correlated with its communication complexity rank  $r(G)$ , such that  $H(G) = \Theta(r(G))$ .

#### Rationale (proposer's reasoning):

Minimal geometric entropy quantifies the information content of a set in terms of its geometric properties. By linking this invariant to communication complexity, the conjecture suggests that the structure and information content of a graph's vertices are closely related to the amount of communication needed to compute a function on those vertices.

**Taxonomy category:** GeometricEntropy (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 60731f1b5cffffa89

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For a given planar graph  $G$ , the conjecture is supported if the correlation coefficient between minimal geometric entropy  $H(G)$  and communication complexity rank  $r(G)$  is greater than or equal to 0.8 for all seeds with at least one seed having a mean correlation coefficient of 0.7.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 1.00       | UNCERTAIN        | SAFE             |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "geometric information theory" AND "communication complexity" AND planar graph" - "vertex set minimal geometric entropy" AND communication complexity rank" - "linear correlation" AND planar graph AND communication complexity

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_planar(n, edges):
        if n < 3:
            return True
        max_edges = 3 * (n - 2)
        if len(edges) > max_edges:
            return False

        # Check for planarity using Euler's formula: V - E + F = 2
        F = len(edges) // 2 + 1 # At least one face (the outer face)
        if n - len(edges) + F != 2:
            return False

        # Check for simple graph (no self-loops or multiple edges between the same pair of vertices)
        edge_set = set()
        for u, v in edges:
            if u == v or (u, v) in edge_set or (v, u) in edge_set:
                return False
            edge_set.add((u, v))

        return True

    def min_geometric_entropy(n, positions):
        # Calculate the minimum number of bits required to encode the positions
        max_bits = 0
        for x, y in positions:
            max_bits = max(max_bits, math.ceil(math.log2(1 / (x - 0.5) * (1 - x + 0.5))))
            max_bits = max(max_bits, math.ceil(math.log2(1 / (y - 0.5) * (1 - y + 0.5))))
```

```

    return max_bits

def communication_complexity_rank(n, edges):
    # Calculate the smallest value of k such that any function f defined on G can be computed
    # This is a simplified version and may not accurately reflect the actual communication complexity
    return len(edges) // 2 + 1

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    if n > 40:
        break

    edges = []
    positions = []

    # Generate a random planar graph with n vertices
    while not is_planar(n, edges):
        edges = [(random.randint(0, n-1), random.randint(0, n-1)) for _ in range(random.randint(1, n))]

    # Calculate the minimal geometric entropy
    positions = [(random.random(), random.random()) for _ in range(n)]
    H_G = min_geometric_entropy(n, positions)

    # Calculate the communication complexity rank
    r_G = communication_complexity_rank(n, edges)

    total_metric_value += H_G * r_G
    instances_tested += 1
    n_max = max(n_max, n)

mean_metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0

return {
    "metric_name": "H(G) * r(G)",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = "mapping_undefined"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07cdb9cb.py", line 101, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07cdb9cb.py", line 75, in run_trial
    H_G = min_geometric_entropy(n, positions)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07cdb9cb.py", line 46, in min_geometric_entropy
    max_bits = max(max_bits, math.ceil(math.log2(1 / (x - 0.5) * (1 - x + 0.5))))
                                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed due to a math domain error, which prevented the computation of correlation coefficients necessary to evaluate the conjecture. | next: Investigate and fix the math domain error in the code to allow for proper computation of the correlation coefficients. Once fixed, re-run the test with valid results.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14171        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12669        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 14364        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 10456        |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 15065        |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17601        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 23949        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16896        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12574        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8823         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146568 ms total latency. Provider mix: {'ollama\_remote': 10}

(full prompt+response transcripts available in `research/audit/04312cc1eeef.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/04312cc1eeef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/04312cc1eeef.tar.gz` (if generated)